

Relatório de Laboratório

Laboratório de Experimentação de Software

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Repositorios populares + Setup do Kanban.md
Grupo (trio)	Brenda Evers · Gabriela Alvarenga Cardoso · Carlos José Gomes Batista Figueiredo
Link do repositório / GitHub Projects	https://github.com/gabialvarenga/lab-experimentacao-software
Data de entrega	20/08/2026

1. Introdução

Este laboratório caracteriza os 1.000 repositórios open-source mais populares do GitHub, buscando entender se popularidade anda junto com outros sinais que costumam ser associados a projetos "saudáveis": maturidade, contribuição externa, cadência de releases, atividade recente, escolha de linguagem, gestão de issues, efeito de rede via forks e práticas de engenharia (licenciamento, CI/CD). O interesse prático é verificar se estrela é um proxy razoável para esses atributos ou se, ao contrário, mede só visibilidade acumulada, independente da saúde atual do projeto.

As questões de pesquisa investigadas, cada uma com sua hipótese informal formulada a partir da leitura das distribuições de *data/data-quality-report.md* (detalhamento completo nas seções abaixo), são:

RQ	Pergunta	Hipótese Informal
RQ1	Sistemas populares são maduros/antigos?	Sustentada: mediana de ~7,7 anos, mas maturidade não é requisito rígido (mín. de 0,01 ano).
RQ2	Sistemas populares recebem muita contribuição externa?	Parcialmente sustentada: mediana de ~766 PRs aceitas, mas 12,3% são outliers que concentram a maior parte da contribuição.
RQ3	Sistemas populares lançam releases com frequência?	Não sustentada como afirmação única: distribuição bimodal, 28% nunca usa Releases, o resto lança com frequência alta.
RQ4	Sistemas populares são atualizados com frequência?	Sustentada para a maioria: mediana de 3 dias sem push com 19,1% de exceções para projetos descontinuados.
RQ5	Sistemas populares são escritos nas linguagens mais populares?	Sustentada: Python, TypeScript e JavaScript no topo, coerentes com o ranking do GitHub Octoverse.
RQ6	Sistemas populares possuem alto percentual de issues fechadas?	Sustentada com folga: mediana de 86% de issues fechadas.
RQ7	Sistemas escritos nas linguagens mais populares recebem mais contribuição	Pendente

	externa, lançam mais releases e são atualizados com mais frequência?	
RQ8	Sistemas populares possuem alto percentual de issues fechadas?	Validável parcialmente: mediana de 6.339 forks, mas a razão fork/estrela original da hipótese não pode ser calculada, por não possuir <i>stargazerCount</i> no dataset.

A leitura consolidada dos oito sinais é que a popularidade por estrelas correlaciona com maturidade e boas práticas de engenharia, mas não as exige. Em cada RQ existe uma minoria relevante de repositórios populares que rompe o padrão: em RQ01, projetos jovens virais; em RQ02, mega-projetos que concentram a contribuição; em RQ03, repositórios de conteúdo sem releases nem PRs; em RQ04, projetos descontinuados que ainda carregam estrelas antigas. Isso reforça a leitura de estrela como indicador de relevância histórica acumulada, não necessariamente de estado corrente do projeto. O detalhamento completo de cada RQ está nas seções abaixo.

2. Contexto

Este relatório documenta o Lab01 da disciplina de Laboratório de Experimentação de Software, na PUC Minas, sob orientação do prof. Danilo Maia, o primeiro dos cinco laboratórios do semestre. Por ser o ponto de partida da disciplina, não há um laboratório anterior a conectar. É este laboratório, ao contrário, que estabelece as convenções reaproveitadas depois: o board Kanban no GitHub Projects e a rotina de snapshots por sprint, iniciada na Issue #13, seguem sendo a mesma infraestrutura de processo usada nos laboratórios seguintes.

O objeto de estudo são os 1.000 repositórios open-source mais populares do GitHub, medidos por número de estrelas. A escolha de estrela como critério de popularidade segue a convenção usual da comunidade GitHub para repositório relevante, e é o eixo em torno do qual todas as questões de pesquisa giram: se esse tipo de popularidade também se traduz em maturidade, contribuição externa ativa, cadência de entrega e práticas de engenharia. Como referência conceitual para linguagem de programação popular, necessária para RQ05, RQ06 e RQ07, o grupo adotou o ranking do GitHub Octoverse, fonte fixada desde a Issue #3 e mantida sem troca até o fim da coleta, para evitar inconsistência entre RQs que dependem do mesmo tipo de comparação.

3. Metodologia

3.1 Principais Desafios

ORIENTAÇÃO: *Relate as dificuldades técnicas e metodológicas reais enfrentadas pelo grupo — não uma lista de trivialidades já resolvidas, e sim decisões difíceis de fato. Exemplos típicos, conforme o laboratório: limite de taxa (rate limit) da API do GitHub ao consultar milhares de repositórios ou workflow runs (Lab01/Lab03); paginação de grandes volumes de dados; ausência de histórico de mudança de status consultável via API no GitHub Projects, exigindo snapshots manuais recorrentes (todos os laboratórios); dificuldade de padronizar katas de dificuldade equivalente e evitar memorização de soluções pela IA (Lab02); ambiguidade na definição operacional de uma métrica, como lead time (Lab03); dados incompletos ou repositórios sem GitHub Actions habilitado (Lab03).*

[conteúdo do grupo — substituir este texto]

3.2 Tomadas de Decisão

- **Limite de WIP da coluna Doing:** A ser preenchido
- **Fonte para linguagem popular:** GitHub Octoverse, escolhida na Issue #3 em vez de TIOBE ou GitHub por já refletir diretamente o comportamento de repositórios hospedados no próprio GitHub, a mesma base de onde vêm os dados coletados, e mantida como fonte única em todas as RQs que dependem desse ranking.
- **Critério de inclusão de repositórios:** a busca usa o filtro stars:>1, excluindo da amostra repositórios com zero ou uma estrela, tipicamente contas de teste ou forks pessoais sem uso real, sem afetar o recorte de top 1.000 por estrelas.
- **Definição operacional da razão de issues fechadas em RQ06:** quando issues.totalCount é zero, a métrica retorna explicitamente 0 em vez de NaN, para não descartar da análise repositórios que simplesmente não usam o rastreador de Issues do GitHub, como o torvalds/linux. O trade-off é que esses casos entram como outlier técnico na distribuição, não como sinal real de má manutenção, distinção já explicitada na hipótese informal de RQ06.
- **Estratégia de coleta em duas fases para chegar a 1.000 repositórios:** foi optado por separar a busca leve, só com os nomes, do enriquecimento em lote, com os campos completos. O custo foi mais chamadas à API no total, mas sem o risco de timeout de gateway que a paginação profunda com campos aninhados apresentava.
- **Strategy Pattern para cálculo de métricas, adotado na Issue #27:** cada métrica, de RQ01 a RQ11, foi isolada como uma implementação de MetricStrategy, desacoplada do script de coleta. O trade-off assumido foi aceitar uma camada extra de abstração, não estritamente necessária para uma coleta pontual, em troca de poder adicionar o RQ08 bônus e as métricas extras da Issue #26 sem alterar o núcleo de coleta já testado.
- **Ampliação do escopo de métricas na Issue #26:** decidido por coletar licença, presença de CI/CD e número de linguagens além do que o enunciado pedia, para poder caracterizar maturidade de engenharia e não só popularidade. Essa ampliação é parte da fatia de inovação detalhada na seção 3.6.

3.3 Etapas

ORIENTAÇÃO: Descreva o processo de desenvolvimento em sprints, seguindo a estrutura do enunciado (ex.: Lab0XS01, S02, S03 + Relatório Final), com o que foi efetivamente entregue em cada uma e quem (qual integrante) foi responsável por qual parte — a correção do professor é feita a partir do board (GitHub Projects), então a divisão aqui deve refletir os Assignees reais das Issues, não uma divisão apenas narrativa. Inclua também a subseção “Configuração do processo” exigida em todos os laboratórios: as colunas do board (mínimo Backlog → To Do → Doing → Review → Done), a política de limite de WIP em uso, e uma captura de tela (print) do board ao final do laboratório, mostrando o fluxo real de trabalho do grupo.

[Tabela ou linha do tempo com Sprint | Entregas | Responsável(is) | Issues (nº)]

Sugestão: insira aqui o print do quadro Kanban (GitHub Projects) mencionado na orientação acima.

3.4 Ferramentas

- **Coleta de dados:** script próprio src/collect.ts, em TypeScript, executado via tsx sobre Node.js 22, consultando exclusivamente a API GraphQL v4 do GitHub. Sem bibliotecas de terceiros para a chamada à API, conforme exigido pelo enunciado. Autenticação por Personal Access Token, lido de .env. Saída em data/repositories.csv.
- **Qualidade de dados:** script próprio scripts/data-quality.ts, criado na Issue #28, que calcula distribuição, outliers pela regra do IQR e valores ausentes por coluna, gerando data/data-quality-report.md. Esse relatório é a base de todas as hipóteses informais do documento.
- **Testes e verificação de tipos:** Vitest para testes unitários, executado por npm test, e o compilador do TypeScript em modo --noEmit para checagem de tipos, executado por npm run typecheck ou npm run build.
- **Integração contínua:** GitHub Actions, configurado em .github/workflows/ci.yml na Issue #29, executando npm ci, npm run typecheck e npm test a cada push e pull request.
- **Processo e gestão:** GitHub Projects v2 como Kanban do grupo, com snapshots do board exportados por sprint via npm run snapshot, script criado na Issue #13, para data/snapshots/lab01-<sprint>.csv.
- **Fonte de referência externa:** GitHub Octoverse, para o ranking de linguagens de programação populares usado em RQ05, RQ06 e RQ07.

3.5 Tabela de Métricas

ORIENTAÇÃO: Construa uma tabela relacionando cada Questão de Pesquisa à métrica correspondente, sua definição operacional exata (a fórmula ou regra de cálculo — não basta o nome) e a ferramenta/fonte usada para coletá-la. Isso é o que garante que o laboratório seja reproduzível por outro grupo. A primeira linha abaixo é um exemplo ilustrativo (baseado no Lab01); substitua pelas RQs e métricas do seu laboratório.

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01 (exemplo)	Idade do repositório	Data atual – data de criação do repositório	Dias	Script GraphQL (API do GitHub)

3.6 Inovações Propostas pelo Grupo (30% da nota)

- **RQ08 bônus, sobre forks:** proposta na Issue #4, foi investigado se sistemas populares têm alto número de forks. Validada parcialmente com o dataset atual, uma vez que a hipótese original era sobre a razão fork/estrela, mas stargazerCount nunca foi incluído em REPO_FIELDS. A análise ficou limitada à magnitude absoluta de forks, uma limitação registrada como possível lacuna para uma coleta futura.
- <<<< A ser incrementado. >>>>

4. Resultados

4.1 Coleta de Dados

ORIENTAÇÃO: *Relate o volume final de dados efetivamente coletado e analisado — não apenas o volume-alvo do enunciado. Informe: quantos itens restaram após os filtros de qualidade (ex.: dos 1.000 repositórios buscados, quantos tinham dados completos; dos repositórios candidatos, quantos de fato usavam GitHub Actions — Lab03); o período coberto pela coleta; quantos trials/execuções foram concluídos dentro do tempo (Lab02); quantos snapshots do Kanban estão disponíveis e desde quando (Lab04/Lab05); outliers ou dados ausentes identificados, e como foram tratados (removidos, mantidos e discutidos à parte, etc.).*

[conteúdo do grupo — substituir este texto]

4.2 Visualização Gráfica

ORIENTAÇÃO: *Para cada Questão de Pesquisa (do enunciado e das RQs de inovação do grupo), inclua ao menos uma visualização que a responda diretamente, com a pergunta enunciada em texto imediatamente antes do gráfico correspondente, eixos nomeados com clareza e a medida de tendência central adequada indicada (mediana costuma ser preferível a média quando há outliers ou distribuição assimétrica — comum em dados de repositórios de software). Use o tipo de gráfico adequado ao tipo de pergunta, conforme a tabela abaixo, e explicita no texto os valores-chave que aparecem no gráfico (não deixe o leitor “adivinhar” o número a partir da figura).*

Tipo de pergunta / dado	Gráfico recomendado
Comparar uma métrica entre categorias (ex.: linguagem, benchmark DORA)	Barras (ranking) — ordenadas por valor, não alfabeticamente
Comparar dois tratamentos no mesmo grupo (ex.: com IA vs. sem IA)	Boxplot pareado ou gráfico de pontos conectados (before/after)
Distribuição de uma métrica numérica (ex.: idade dos repositórios)	Histograma ou boxplot
Relação entre duas métricas numéricas (ex.: RQ07 do Lab01, RQ05 do Lab03)	Gráfico de dispersão (scatter plot)
Evolução ao longo do tempo (ex.: cycle time por sprint)	Linha, com um ponto por sprint/snapshot
Composição/fluxo do Kanban ao longo do tempo (Cumulative Flow Diagram)	Área empilhada (uma camada por coluna do board)

Proporção de categorias (ex.: % de issues fechadas)	Barra única 100% ou barras simples — evite pizza com muitas fatias
---	--

[Insira aqui os gráficos do grupo, um por RQ, cada um precedido da pergunta que ele responde]

4.3 Discussão

ORIENTAÇÃO: Para cada RQ (do enunciado e das RQs de inovação do grupo), compare explicitamente a hipótese informal levantada na Introdução com o resultado efetivamente obtido — hipótese confirmada, refutada, ou parcialmente confirmada, e por quê. Quando houver teste estatístico (ex.: Wilcoxon no Lab02), reporte o valor obtido e interprete o que ele significa em linguagem acessível, não apenas o número bruto. Discuta as ameaças à validade específicas do laboratório (ex.: efeito de aprendizado entre katas e risco de memorização pela IA — Lab02; diferença de dificuldade entre laboratórios distintos confundindo a tendência de cycle time — Lab05; lacunas nos snapshots do Kanban — Lab05). Finalize relacionando o que as inovações do grupo (seção 3.6) acrescentaram: elas confirmaram, contradisseram ou aprofundaram o que os 70% do enunciado já mostravam?

[conteúdo do grupo — substituir este texto]

5. Conclusão

ORIENTAÇÃO: Sintetize, em poucos parágrafos, as respostas a todas as RQs (enunciado + inovação do grupo), sem repetir números já discutidos em detalhe — o objetivo aqui é a mensagem final, não os dados brutos. Aponte as principais limitações do estudo (tamanho de amostra, ameaças à validade não mitigadas, período de coleta). Quando o enunciado pedir explicitamente uma postura de consultoria (caso do Lab05, que pede recomendações de melhoria de processo “como se o grupo fosse consultoria para um time real”), inclua recomendações objetivas e acionáveis, não genéricas. Encerre indicando o que o grupo faria diferente com mais tempo ou recursos, e quais das inovações propostas (30%) valeriam a pena expandir em um trabalho futuro.

[conteúdo do grupo — substituir este texto]

5. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
-